Q-1: Valid Elements in a Topological Ordering

You are given a **Directed Acyclic Graph (DAG)** with N vertices (1 to N) and M edges. There is a directed edge from *edges[i][0]* to *edges[i][1]* in the given *edges* array.

As you may know, a given DAG may have multiple **topological orderings**.

A **topological ordering** is a linear ordering of its vertices such that for every directed edge from vertex u to vertex v ($u \rightarrow v$). u is listed before v.

You are provided with a valid **topological ordering** *topoSort* of the given DAG.

We call a vertex u **valid** if we can use that vertex in the topological ordering and it has not been used before in the ordering.

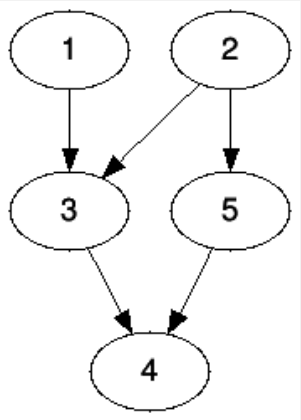
Your task is to find out the **number of valid vertices before every stage of the given topological ordering**. (See the example test case for more clarity.)

The results are stored and returned in a vector of size N.

Expected Time Complexity: $\Theta((N + M)log(N))$

Example Test Case:

TEST CASE 1
Input: `N = 5, M = 5`
`edges = [[1,3],[2,3],[3,4],[2,5],[5,4]]`
`topoSort = [1,2,5,3,4]`
Output: `[2,1,2,1,1]`
Explanation:



- In the start, only vertex 1 and 2 are valid. *No. of valid elements = 2*
- After vertex 1 is used in the ordering, only vertex 2 is valid. *No. of valid elements = 1.*
- After vertex 2 is used in the ordering, vertex 3 and 5 are valid. *No. of valid elements = 2.*
- After vertex 5 is used, only vertex 3 is valid. *No. of valid elements = 1.*
- After vertex 3 is used, only vertex 4 is valid. *No. of valid elements = 1.*

Input Format:

1. The first line contains two integers N and M - the number of vertices in the DAG and the number of edges in the DAG.
2. The next M lines contains M elements of the array *edges*.
3. The third line contains N elements of a valid topological ordering of the above DAG.

Output Format: Return a vector of size N.

`You are free to use any inbuilt function`

Q-2: Minimum Cost to provide water

Given an integer N, representing the number of villages numbered from 0 to $N-1$, M representing the number of pipes, an array `wells[]` where `wells[i]` denotes the cost to build a water well in the i-th village, and a 2D array `pipes` in the form of $[X, Y, C]$ which denotes that the cost to connect village X and village Y with a water pipe is C. Your task is to provide water to each and every village, ensuring that each village either has its own water well or is connected to some other village that has water. Find the minimum cost to accomplish this.

Example Test Cases:

TEST CASE 1
Input:`N = 3, M = 2, wells = [1, 2, 2], pipes= [[0 1 1], [1 2 1]]`
Output:`3`
Explanation:
Build well in village0 to provide water with cost=1
Connect village1 and village0 with cost =1 , now village0 and village1 have water.
Finally connect village2 with village1 with cost=1 , now all the villages have water with total cost=3.

TEST CASE 2
Input:`N = 4, M = 3, wells = [1, 1, 1, 1], pipes = [[0 1 100], [1 2 100], [1 3 50]]`
Output:`4`
Explanation:
Clearly its better to construct well at each village rather than building costly roads. Hence total cost=1+1+1+1=4.

Expected Time Complexity: $O(E \cdot \log \mathrm{N})$ where E is the number of edges and N is the number of vertices in the formed graph.

Input Format:

1. The first line contains T, the number of testcases.
2. The first line of each testcase contains two integers N and M - the number of villages and the number of pipes.
3. The second line of each testcase contains a list of integers wells[] where wells[i] represents the cost to build a well in the ith village.
4. Next M lines of each testcase will contain three integers X Y C where X is the village number connected to village Y, Y is the other village connected to village X, C is the cost to connect the villages X and Y with a pipe.

Output Format:
A single integer representing the minimum cost to provide water to every village.

`You are free to use any inbuilt function`

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)